Large-scale Neural Network Quantum States for *ab initio* Quantum Chemistry Simulations on Fugaku

1st Hongtao Xu
School of Advanced Interdisciplinary Sciences
University of Chinese Academy of Sciences
State Key Lab of Processors
Institute of Computing Technology, CAS
Beijing, China
xuhongtao22@ict.ac.cn

2nd Zibo Wu
College of Chemistry
Beijing Normal University
Beijing, China
202231150051@mail.bnu.edu.cn

3rd Mingzhen Li
State Key Lab of Processors
Institute of Computing Technology, CAS
Beijing, China
limingzhen@ict.ac.cn

4th Weile Jia
State Key Lab of Processors
Institute of Computing Technology, CAS
Beijing, China
jiaweile@ict.ac.cn

***Abstract*—Solving quantum many-body problems is one of the fundamental challenges in quantum chemistry. While neural network quantum states (NQS) have emerged as a promising computational tool, its training process incurs exponentially growing computational demands, becoming prohibitively expensive for large-scale molecular systems and creating fundamental scalability barriers for real-world applications. To address the above challenges, we present QChem-Trainer, a high-performance NQS training framework for *ab initio* electronic structure calculations. First, we propose a scalable sampling parallelism strategy with multi-layers workload division and hybrid sampling scheme, which break the scalability barriers for large-scale NQS training. Then, we introduce multi-level parallelism local energy parallelism, enabling more efficient local energy computation. Last, we employ cache-centric optimization for transformer-based *ansatz* and incorporate it with sampling parallelism strategy, which further speedups up the NQS training and achieve stable memory footprint at scale. Experiments demonstrate that QChem-Trainer accelerate NQS training with up to 8.41x speedup and attains a parallel efficiency up to 95.8% when scaling to 1,536 nodes.**

***Index Terms*—Neural network quantum state, Scientific computing, Many-body Schrödinger equation, Massively parallel processing**

1. Introduction

Efficiently solving the electronic Schrödinger equation has long been a key challenge for quantum many-body physics and quantum chemistry. Various standard methods have been proposed for solving the electronic Schrödinger equation such as full configuration interaction (FCI) [1], coupled cluster (CC) [2], and second-order Møller–Plesset perturbation (MP2) theory [3]. However, due to the exponential growth of computational complexity with the electronic degrees of freedom, only a limited number of systems can be solved exactly. In addition to these deterministic methods, variational Monte Carlo (VMC) approach [4], which optimizing trial wavefunctions through stochastic sampling, provides an effective alternative. Recently, the neural network quantum states (NQS) [5], [6] have emerged as a powerful tool to handle the quantum many-body problem. NQS tries to overcome the curse of dimensionality inherent in many-body quantum systems by leveraging the expressive power of neural networks to parametrize quantum state with neural network. In 2017, Carleo and Troyer first utilize restricted Boltzmann machines (RBMs) to describe prototypical interacting spin models [5]. Various deep learning architectures, including recurrent neural networks (RNNs) [7], [8],convolutional neural networks (CNNs) [9], [10] and transformers [11], [12], have been integrated into NQS to further improving the accuracy to quantum many-body systems. Among them, transformer-based architecture exhibits huge potential to capture complex correlations in quantumn chemistry systems [11], [13].

Despite rapid developments, NQS training still faces several challenges especially when applied to large-scale molecular systems. First, the sampling process of NQS training involves enormous samples, leading to substantial computations and memory footprints. Additionally, the auto-regressive property of sampling incurs dynamically increased amount of samples, resulting significant scalability challenges when extending NQS to modern HPC systems. Worse still, due to the exponential growth of complexity of Hamiltonian, the local energy calculation exhibits prohibitively expensive computation cost, hendering the simu-

lation of large-scale molecular systems. Last but not least, the complexity of transformers also grows quadratically and incurs enormous memory requirement such as Key/Value cache, which is directly proportional to the electron orbitals and number of samples in NQS training.

To address the above challenges, we propose QChem-Trainer, an innovative high-performance NQS training framework for *ab initio* electronic structure calculations and adapt it to the supercomputer Fugaku [14]. The major innovations of this work can be summarized as follows.

- A scalable and efficient sampling parallelism for NQS training, which incorporates multi-stage workload partitioning, density-aware dynamic load balance strategy and memory-stable hybrid sampling scheme. Our parallelism overcomes the scalability barrier and efficiently handles the dynamic memory consumptions inherent in NQS algorithm.
- A multi-level energy calculation parallelism. We fully exploits the parallelism inherent in Hamiltonians through a hybrid parallelization, benefiting not only the NQS training but also other simulations in classical *ab initio* quantum chemistry.
- Cache-centric optimization for transformers based *ansatz*. To handle the linearly increased cache size with the number of samples, we propose a dynamic cache management strategy which employs fixed-size cache pooling, selective recomputation and movement optimizations. We incorporate it into our sampling parallelism and achieves stable memory consumptions.
- Case studies of real chemical systems demonstrate the effectiveness and scalability of our framework on the top supercomputers such as Fugaku.

Although our implementation is based on Fugaku, our optimization is architecture independent except the vectorization of Hamiltonians calculation. We believe QChem-Trainer can be easily transplanted to other systems, such as GPU systems. Our work opens the door for unprecedentedly large-scale NQS for *Ab initio* quantum chemistry .

2. Background

2.1. Variational Monte Carlo (VMC) Algorithm

Given the variational wavefunction *ansatz* $|\Psi_\theta\rangle$, the ground state energy E_θ can be written as:

$$\langle E\rangle = \frac{\langle\Psi_\theta|\hat{H}|\Psi_\theta\rangle}{\langle\Psi_\theta|\Psi_\theta\rangle} = \frac{\sum_n |\langle\Psi_\theta|n\rangle|^2 \frac{\langle n|\hat{H}|\Psi_\theta\rangle}{\langle n|\Psi_\theta\rangle}}{\sum_n |\langle\Psi_\theta|n\rangle|^2} = \mathbb{E}_{P_\theta}[E_{\mathrm{loc}}(n)] \tag{1}$$

Here the θ represents the parameters need to be optimized through VMC algorithm, which is equivalent to nerual network optimization problem in NQS. $\hat{H}$ represents the second quantized Hamiltonian in quantum chemistry, which can be written as:

$$\hat{H} = \sum_{pq} h_{pq}\hat{a}_p^\dagger\hat{a}_q + \frac{1}{4}\sum_{pqrs}\langle pq\|rs\rangle\,\hat{a}_p^\dagger\hat{a}_q^\dagger\hat{a}_s\hat{a}_r \tag{2}$$

The matrix element H_{nm} in $\hat{H}$ is defined as $\langle n|\hat{H}|m\rangle$ where the $|n\rangle$ is the occupation number vector (ONV). Specifically, the ONV $|n\rangle$ is expressed as $|n_{1\alpha}, n_{1\beta}, \ldots, n_{K\alpha}, n_{K\beta}\rangle$ with $n_{k\sigma} \in \{0, 1\}$, where $n_{k\alpha}$ and $n_{k\beta}$ represent the occupation number of the *alpha* and *beta* spin orbital with the same spatial orbital, respectively. The probability distribution $P_\theta(n)$, which can be obtained using Markov Chain Monte Carlo (MCMC) Sampling, can be expressed as:

$$P_\theta(n) = \frac{|\langle\Psi_\theta|n\rangle|^2}{\sum_n |\langle\Psi_\theta|n\rangle|^2} \tag{3}$$

As shown in (1), the ground state energy can be evaluated using the average of the local energy. Furthermore, we could evaluate the gradient of the energy as follows:

$$\partial_\theta\langle E\rangle = 2\Re\big[\big\langle(\partial_\theta \ln\Psi^*(n))\big\rangle(E_{\mathrm{loc}} - \langle E\rangle)\big\rangle_n\big] \tag{4}$$

where $\partial_\theta \ln\Psi^*(n)$ is the gradient of the parameters θ, which can be computed using the standard automatic differentiation techniques. The parameters θ are updated based on the $\partial_\theta\langle E\rangle$ with appropriate optimizer, such as SGD, Adam, AdamW and K-FAC [15]–[18].

2.2. NQS Workflows

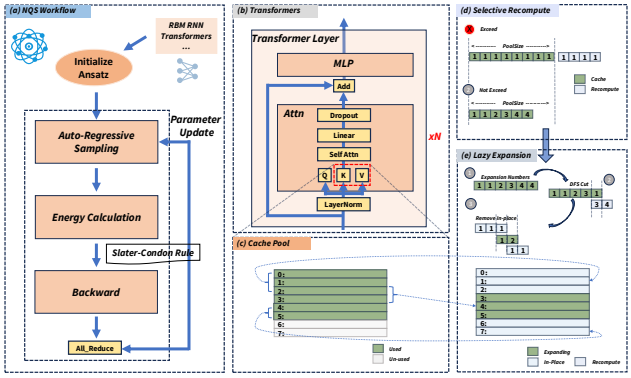


Figure 1: (a): NQS workflows. (b): Transformers architecture. (c-e): Cache-centric optimization.

As shown in Fig. 1a, the NQS workflows comprise three components: auto-regressive sampling, energy calculation and backward. In the beginning, NQS initialize the wave function *ansatz* with a customized neural network, such as RBM, RNNs or Transformers. Then, NQS optimize the neural network iteratively. First, NQS employs sampling phase to get the energy estimation at current state, which mirrors the inference of neural network. Next, NQS compute the energy estimations of the obtained samples, which involves substantial calculation of Hamiltonians. Last, as shown in (4), NQS evaluate the approximate gradient by minimize the ground state energy, followed by optimizer step to update parameters.

In sampling phase, NQS mirrors the network inference except for the total probability property. Unlike traditional

language models decoding, which predict a probability distribution and select the most probable token as the next output, NQS sampling phase take all the probability into account. Specifically, NQS conducts stochastic sampling with a fixed number of samples (N_{count}) under a probability distribution given by one step neural network inference. This approach theoretically leads to an increase in sample size by a factor of $O(V^N)$, where N is sample steps and V denotes the probability table size, typically represents four possible occupation patterns corresponding respectively to the states $\{|\text{vac}\rangle, |\alpha\rangle, |\beta\rangle, |\alpha\beta\rangle\}$. Furthermore, chemical-informed prouning is employed to remove the invalid states [19] during every sampling steps.

3. Innovations

3.1. Scalable and Memory-stable Sampling Parallelism

In practice, the number of unique sample (N_u) represents the actually workloads during each sampling step (similar to the batch size of nerual network inference). However, as mentioned in Section 2.2, NQS sampling phase faces exponential increased samples, leading to vast N_u and huge memory requirements. Worse still, due to the parameter evolution and the randomness of inherent in NQS sampling, N_u is dynamic changed during each iteration. Those characteristics incur huge and unpredictable memory requirements, posing significant barriers for large-scale NQS training.

To address those challenges, we propose a scalable and memory-stable sampling parallelism, which comprises three core components: (i) Multi-stage workload partitioning, which overcomes the barrier of scaling to massively parallel systems. (ii) Density-aware load balance, which tackles the load imbalance problem caused by the dynamic change of N_u through historical information. (iii) Hybrid sampling strategy, which combines breadth-first search (BFS) and depth-first search (DFS) sampling schemes to constrain the peak memory consumptions and enhance the stability during large scale training.

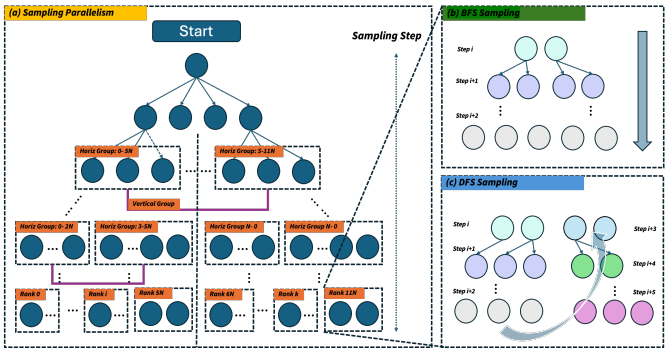


Figure 2: Overview of the sampling parallelsim. (a): Multi-stage workload paritioning. (b): Default BFS sampling scheme. (c): DFS sampling scheme.

3.1.1. Multi-stage Workload Partitioning. As shown in Fig. 2a , the NQS sampling phase can be regarded as a quadtree, where the tree layers and nodes represent auto-regression sampling steps and unique samples, respectively. We can simply implement an embarrassingly parallel approach where N_p processes samples independently. However, this approach introduces significant redundant samples, resulting in low end-to-end training efficiency. A straightforward solution to the redundancy issue is to fix the random seed, ensuring that each process generates an identical quadtree. Subsequently, the tree nodes at a specific $k-th$ layer are parallelized [11]. However, this method still faces a scalability barrier in large-scale parallelization. The root cause lies in its inherent requirement for all processes to sample the entire set of unique samples (N_u) at the $k-th$ layer before distributing the workloads. N_u must satisfy $N_u \geq cN_p$ to achieve load balance across the processors, where c is a constant. As N_p increases, N_u grows proportionally, quickly exceeding the memory capacity of a single device. This limitation makes this one-stage partitioning method unable to scale effectively on modern supercomputers.

To address this problem, we propose a multi-stage workload partitioning strategy for NQS, as illustrated in Fig. 2a . Contrast to one-time workload division, our method organizes processes into multiple groups and gradually partitions the workload within each group. This design effectively mitigates the challenges posed by the large N_u requirement. Specifically, given a list of process group sizes G_n and a list of split layers L, we divide the current workload into $G_n[i]$ parts at layer $l[i]$, where i is the index for G_n and L. The relationship between the total number of processors (N_p) and the partition setting (G_n) satisfy $N_p = \prod_{i=1}^{|G_n|} G_n[i]$.

To facilitate this partitioning and density-aware load balance (see in Section 3.1.2), we organize the processes into VerticalGroups V_g and HorizGroups H_g at each partition layer. As depicted in Fig. 2a, VerticalGroups V_g is responds to partition while the HorizGroups is for load balancing, detailed in §3.1.2. When current layer k is arrive on split layers $L[i]$, every ranks apply division across $V_g[i]$ according to weights (detailed in §3.1.2). For example, given the $G_n = [2, 2, 3]$ (as illustrate in Fig. 2a, the purple line reference vertical group) and $L = [6, 8, 10]$. In rank 0, $V_g = [[0, 6], [0, 3], [0, 1, 2]]$ and $H_g = [[0, 1, 2, 3, 4, 5], [0, 1, 2], [0]]$. This hierarchical grouping enables efficient workload distribution and load balance across massively parallel environment.

3.1.2. Density-aware Dynamic Load Balancing. The inherent unpredictability of sampling results complicates accurate workload prediction during intermediate layer partitioning. Traditional static partitioning methods, which rely on the known metrics in current layer like unique sample counts or total samples, often result in significant load imbalance, especially at large scales. We address this challenge through a density-aware dynamic load balancing strategy founded on two key observations: (i) Continuity of parameter evolution Neural network parameters evolve gradually through incre-

Algorithm 1 Process Group Initialization

Given current HorizGroup H_{go}, split list L
Return H_g, V_g
$H_g, V_g = [], []$
$H_{go} = get\_default\_group()$
for $i = 0, \ldots, len(L) - 1$ **do**
    $ws = H_{go}.get\_wordsize()$
    $rank = H_{go}.get\_rank()$
    $tmp \leftarrow \lfloor rank/L[i] \rfloor$
    $H_g.append(init\_group(range(tmp * L[i], (tmp + 1) * L[i])))$
    $V_g.append(init\_group(rank\%L[i] + L[i] * range(ws/L[i])))$
    $H_{go} = H_g[-1]$
end for

mental updates during training, resulting in smooth variations in sample distributions between consecutive iterations. This continuity allows for predictable workload patterns. (ii) Spatial locality in quantum states chemically valid configurations exhibit spatial correlation in the sampling quadtree structure due to molecular orbital interactions.

We leverage these properties through a novel density metric (d) representing the *unique-to-sample ratio* ($d = \frac{sample\_unique}{sample\_counts}$). This metric enables workload prediction by multiple sample counts (known in current layer) and d. As shown in Algorithm 2 (Lines 6-12), we refine the original static load balance by incorporating this dynamic metric with minimal overhead from local communication within H_g. Our hierarchical group organization (Fig. 2) enables efficient implementation, as discussed in §3.1.1.

Algorithm 2 Multi-stage Workload Partition with Density-aware Load-balance

1: **Given** $L, G_n : List[int]; Rank : int; D = 1 : int$
2: **Return** $sample\_unique$, $sample\_counts$
3: $V_g, H_g \leftarrow InitGroup(Rank, G_n)$
4: **for** $i = 0, \ldots, len(L) - 1$ **do**
5: $sample\_unique, sample\_counts \leftarrow Sampling(L[i], L[i + 1])$
6: $w \leftarrow sample\_counts$
7: $D_{avg} \leftarrow AllReduce(D, H_g[i])$
8: $D\_lst \leftarrow AllGather(D_{avg}, V_g[i])$
9: $p\_idx \leftarrow Partition(sampe\_counts, V_g[i])$
10: **for** $j = 0, \ldots, len(p\_idx) - 1$ **do**
11: $w \leftarrow w[p\_idx[j] : p\_idx[j + 1]] * D\_lst[j]$
12: **end for**
13: $sample\_unique, sample\_counts \leftarrow Partition(w, V_g[i])$
14: **end for**
15: $Density \leftarrow \frac{sample\_unique}{sample\_counts}$

3.1.3. Memory-stable Hybrid Sampling Strategy. To address the challenge of memory instability caused by the combinatorial explosion of samples ($O(V^N)$), we propose a hybrid BFS-DFS sampling strategy with guaranteed memory constraints. As illustrated in Fig. 2(b-c), this method integrates breadth-first search (BFS) and depth-first search (DFS) approaches to optimize memory usage and ensure stable performance during large-scale training. Our proposed method enables automatic switching between BFS and DFS sampling schemes based on the current sample count N_u relative to the threshold . Specifically, when $N_u < k$, sampling proceeds at the granularity of tree layers, similar to traditional BFS. Once $N_u \geq k$, the sampling process switches to a DFS approach with a stride of k, effectively managing peak memory consumption by processing smaller batches sequentially. In this mode, a stack is maintained to track the string information of samples exceeding the threshold, discarding intermediate caches to further reduce memory footprint. Upon reaching leaf nodes in the quadtree, final sample results are recorded, and the stack is popped to continue the sampling process until completion.

3.2. Multi-level Energy Calculation Parallelism

The evaluation of local energies ((5)) constitutes the computational bottleneck in NQS training particularly for molecular systems with complex second-quantized Hamiltonians. The Hamiltonian element contains $\mathcal{O}(N^4)$ terms for N spin orbitals, resulting in quartic scaling of computational cost with molecular system size.

$$E_{\text{loc}}(n) = \sum_m \langle n| \hat{H} |m\rangle \frac{\Psi(m)}{\Psi(n)} \tag{5}$$

To address this challenge, we develop a multi-level parallelization approach and reform the local energy calculations as Algorithm 3. We first decompose the energy calculation as (6). First, we parallelize the each ONV $|n\rangle$ terms in MPI-level, which is the unique samples N_u obtained from NQS sampling phase and corresponds to the outer loop in (6) . Then, we employs threads-level parallelization to further accelerate the calculation, which is the middle loop in (6). Last, to fully exploit the parallelism, we vectorize the inner loop in SIMD level.

$$E_{\text{loc}} = \frac{1}{N_p} \sum_{k=1}^{N_p} \left(\sum_{n \in \mathcal{N}_k} \left(\sum_m \langle n| \hat{H} |m\rangle \frac{\Psi(m)}{\Psi(n)} \right) \right) \tag{6}$$

However, the vectorization of the matrix element computation $\langle n| \hat{H} |m\rangle$ in inner loop is not trivial. There lies three key barriers: (i) The number of orbits in chemical system always not fit into a vector register and the number of orbits always differs across different systems. (ii) Conditional branching inherent in Slater-Condon rules [20], which is our baseline implementation and require three excitation conditions: identical configurations, single excitations, and double excitations. (iii) The complex bit-wise operations and irregular access patterns for two-electron integrals (h2e) arrays.

We overcome these barriers through three optimizations:

- **Qubit-packing**: We compress orbital occupations into 64-qubit chunks so that each chunk can be stored as a double float format. Consequently, through interleaved loading double floats into multiple vector registers and carefully dealing with the interaction of those vector registers, we can fully exploit the parallelism in SIMD level. Algorithm 3 line 6-7 demonstrate one chunk situation.
- **Branch Elimination**: In practical chemical systems, the double excitation state dominates the calculation. Based on this chemistry insight, we remove the conditional branches for more thorough vectorization and

efficient execution pipeline and implement customized check function to ensure the correctness (see Algorithm 3 line 12-14). In this case, some common calculation can be fused such as creations (annihilations) state counting and h2e access (see Algorithm 3 line 8-11).

- **Other SIMD Patterns**: Implement other frequently used component, including parity calculation (see Algorithm 3 line 9).

Experiment shown in §4 demonstrate our methods is highly efficient and scalable.

Algorithm 3 Three-level Parallelism in Energy Calculation

```
1: Partition samples across MPI processes: M = ⋃_{k=1}^{N_p} M_k
2: Each process:
3: for n ∈ N_k in OpenMP parallel do                ▷ Thread-level Paralelism
4:     Compute ⟨n| Ĥ |m⟩ via SVE-vectorized:
5:     for i = 0 to N_vec with stride svcnt do
6:         sv_bra ← sv_dup(n)                       ▷ Broadcast bra state
7:         sv_ket ← svld1(m[i])                     ▷ Load svcnt ket state
8:         sv_p, sv_q, sv_n ← sv_fused_bitop(sv_bra, sv_ket)
9:         sv_sgn ← sv_parity(sv_bra, sv_ket, sv_p, sv_q)
10:        sv_Hni ← sv_mul(sv_sgn, sv_h2e(h2e, sv_p, sv_q))
11:        sv_store(sv_Hni)
12:        pred_0 ← sv_cmpeq(sv_n, 0)               ▷ Identity other case
13:        if sv_ptest_any(pred_0) then
14:            Customized_function()
15:        end if
16:    end for
17: end for
18: Reduce local energies: MPI_Allreduce(E_loc^(k))
```

3.3. Cache-Centric Optimization for Transformers

Key/Value cache (KVCache), which stores previous Key/Vaules to avoid redundancy computations when decoding, is the common optimization technique [21] to accelerate the inference of transformer architectures. However, directly applying KVCache to NQS sampling is infeasible. Specifically, the size of KVCache is in direct proportion to the N_u and its length (the sampling steps). In practical NQS training, N_u is required to be very large (often exceeds 10^6) for accurate energy estimation, leading to substantial memory requirements. Furthermore, the dynamic change of N_u in each iteration also increases the uncertainty in memory footprint. Worse still, within the perspective of sampling step, N_u is expanded rapidly with roughly $O(N_{step}^4)$ scaling, resulting in frequent data movements. Therefore, the memory issues of KVCache hinders its applicability in large scale molecular systems. To address this problem, we propose a dynamic cache management strategy which employs cache pooling, selective recomputation and lazy cache expansion, achieving stable memory footprint and speedup within limited memory capacity.

3.3.1. Fixed-size Cache Pooling and Selective Recomputation. We first organize the KVCache into a fixed memory pre-allocation pool. Cache pooling strategy not only ensure the controllable peak memory consumption but also avoid the overhead of frequent memory allocations and segmentation caused by samples expanding. Additionally, selective recomputation is employed to handle the situation when the unique samples exceed the cache pool capacity. When the KVCache exceed the capacity, we discard the exceeding part of KVCache and employ recomputation strategy to get the previous Key/Value. Although recomputation increases the computation FLOPs, its side effect remains negligible especially when incorporates with hybrid sampling strategy (detailed in Section 3.1.3). Specifically, we reuse the sampling chunk size k as cache line size. The recomputation will be employed only in case of switching to DFS sample scheme, which the sampling chunks' KVCache will be discarded except for the first one. Therefore, our strategy only increase once additional computation in sampling scheme switching layer.

3.3.2. Lazy Cache Expansion. We optimize the data movement through lazy cache expansion during samples increasement in NQS training. Due to the property of sampling, each sample step the batch size will expand with the fact of λ (≤ 4), leading memory reorder in the cache pool. We optimize this process by delaying the KVCache expansion and seeking the underline optimization opportunity, which we named it lazy expansion. As shown in Fig. 1(c-d), we just keep the expanding indexs after generating the new unique samples. We optimize the unnecessary memory reordering by analysis the indexs: (i) Detecting over-long expansion by sampling chunk size k (ii) Keep in-place of the leading cache block which expanding size is 1 (iii) Directly in-place moving the least cache block which expanding size is non-zero.

4. Evaluation

4.1. System Architecture and Evaluation Setup

We implement QChem-Trainer on Fugaku supercomputer, which comprises 158,976 nodes with a theoretical peak performance of 537 PFLOPS and currently ranks No.6 and No.1 on the TOP500 list [14] and HPCG benchmark [22], respectively. Each computing node is equipped with one A64FX SoC, which consists of 52 cores with 4 dedicated for OS and 48 for computation. The A64FX SoC is organized into four Core Memory Group (CMGs), with each CMG connected to 8GBs local HBM2 memory with 256GB/s bandwith. Moreover, the A64FX architecture supports 512-bit SVE operations, enabling each A64FX SoC to reach a theoretical peak performance of 3.38 TFLOPS at 2.2GHz.

In evaluation, we use transformers as the wavefunction ansatz for equal comparation. The setting is as the follows. For the amplitude part of wavefunction, we use 8 decoder-only layers with number of attention head $n_{head} = 8$ and hidden size $d_{model} = 64$ [23]. For phase, we use three layers MLP with sizes $N*512*512*1$, where N is the number of spin orbits. We use AdamW [24] as optimizer and the learn

rate is $lr = 1e-2$ which schedules as (7) with $n_{warmup} = 2000$ for default.

$$\eta_t = d_{\text{model}}^{-0.5} \times \min\left((t+1)^{-0.5},\ t \times n_{\text{warmup}}^{-1.5}\right) \quad (7)$$

All the experiments are performed on Fugaku with a NUMA-aware MPI setting.

4.2. Precision and Performance Validation

We evaluate our optimization precision within three real-world molecular systems: N_2, PH_3 and LiCl and they are all in STO-3G basis set. As shown in Table 1, QChem-Trainer can reach the same the ground state energies compared to existing accurate results such as FCI. Additionally, we calculate the potential energy surface of the N_2 molecular system for more comparison, which is shown in Fig. 3 Left. Results demonstrate our optimizations precision.

TABLE 1: Ground state energies (in Hartree) calculated by our method. The HF, CCSD and FCI results are shown for comparison. N is the number of qubits and N_e the total number of electrons (including spin up and spin down).

			Energy (Hartree)			
Molecule	N	N_e	HF	CCSD	Ours	FCI
N_2	20	14	-107.4990	-107.6560	-107.6602	-107.6602
PH_3	24	18	-338.6341	-338.6981	-338.6984	-338.6984
$LiCl$	28	20	-460.8273	-460.8475	-460.8494	-460.8496

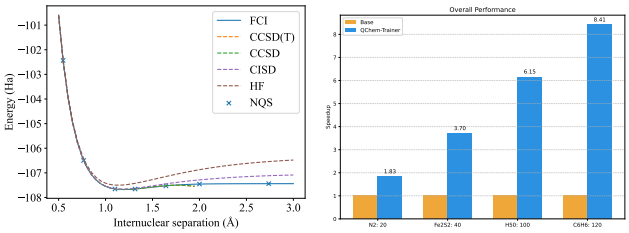


Figure 3: Left: Potential energy surfaces of N_2. Righ: Overall speedup. Four molecular systems are chosen to test QChem-Trainer's performance. From left to right, the spin orbits in molecular system increases. The figure demonstrate the normalized speedups compared to baseline implementation.

To test the overall performance gains, we choose four systems with the increased spin orbits: N_2, Fe_2S_2, H_{50} and C_6H_6 with 20, 40, 100, 120 spin orbits respectively. Among them, Fe_2S_2 is a strongly correlated system $[Fe_2S_2(SCH_3)_4]^{2-}$ with CAS(30e, 20o), while the hydrogen chain H_{50} is configured with a bond length $2.0a_0$ using orthonormalized atomic orbitals obtained in STO-6G basis. C_6H_6 is benzene molecular system in the 6-31G basis set.

We illustrate the normalized end-to-end speedup up measured by the execution time per iteration. As shown in Figure 3 Right, C_6H_6 system attains the maximal speedup (8.41x). N_2 system exhibits minimal acceleration with 1.83x speedup. An average 4.95x speedups are achieved over those systems, accelerating the NQS simulation significantly. Also, we found the systems who have more orbits always exhibit more significant performance gains, which we attribute to the more computational complexity in sampling and Hamiltonian calculations.

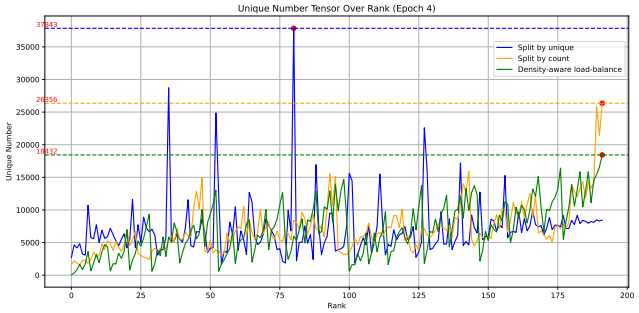
4.3. Case study

In this section, we conduct some case studies in some real chemical systems. We choose H_{50} and Fe_2S_2 to test our sampling parallelism and density-aware load balance, respectively. For energy parallelism, we use N_2, Fe_2S_2 and H_{50} to test step by step speedup with 20, 40 and 100 qubits, respectively. We use two methods of energy calculations, which is sample space calculation and accurate calculation, to test the weak scaling of our methods on 1,536 nodes.

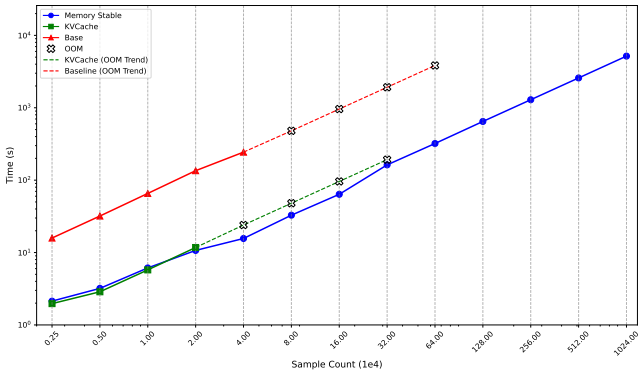
4.3.1. Sampling Parallelism. As shown in Fig. 4b, we compare three sampling method: (i) baseline implementation without KVCache optimization; (ii) KVCache version without hybrid sampling scheme and cache managements; and (iii) Memory-stable version which combines hybird sampling and our cache-centric optimization. We incrementally increase the number of samples from an initial 2.5×10^3 up to 1.024×10^7 to test peak memory footprints and scalability of different methods. The results demonstrate that the KVCache implementation accelerates the sampling process significantly. However, the memory issue hinders its applicability. We can see the KV cache version occurs OOM error in 2×10^4 while the base version in 4×10^4. In contrast, our memory-stable methods can constrain the peak memory footprint and scale up to 1.024×10^7 samples (three orders of magnitude larger) while maintain efficient performance.

4.3.2. Load balancing. We collecte the final obtained unique samples to investigate the load balance problem. We use strongly correlated system Fe_2S_2 system using 256 ranks and test three methods using the same training settings. Therefore, we can use the maximum unique samples across ranks to demonstrate the real workload situation in NQS training. Experiment results are shown in Fig. 4a. We test two static workload division methods (splited by unique sample and by sample count) and ours density-aware load balancing method, which have 37843, 26356 and 18432 maximum unique samples across all ranks respectively. As the blue line shown, evenly distributing the unique samples leads to severe load imbalance problem. Additionally, division along sample counts (the yellow line in Fig. 4a) mitigate the problem. In contrast, our density-aware load balancing method (The green line) showcase the approximately load balance, enabling the large scale NQS training.

4.3.3. Energy Parallelism. We choose three molecular systems: N_2, Fe_2s_2 and H_{50} with increased number of spin orbits to test the step by step speedup of our energy calculation optimization within a single rank. As shown in Fig. 5, we sequentially enable SIMD (implemented by ARM



(a) This figure demonstrates the final obtained unique samples N_u in each rank. The blue line denotes splitting by unique samples while the yellow line denotes splitting by sample counts and the green line represents our desity-aware load-balance strategy.



(b) This figure demonstrates the iteration time under the different sample counts. The read line denotes the baseline implementation. The green line denotes directly using KVCache. The blue line represent our memory-stable approach combining both hybrid sampling scheme and cache management. The white cross denotes Out-of-memory (OOM) error.

SVE extension) and thread-level parallel (implemented by OpenMP). We can observe the up to 20.8x speedup for H_{50} when enable both parallelism (version base+sve+omp).

4.3.4. Scalability. We test the scalability of H_{50} system with the setting of $N_u = N * 4 * 10^3$ for N nodes. Different Ψ calculation mothods (shown in (5)) are tested for comparision. In sample space methods (shown in Fig. 6a), the overhead of construction of LUT is becoming innegligible when the sample numbers increasing, which is setted to avoid redundant Ψ calculation and noted with green colour, thus degrading scalability. For comparision, we remove the LUT and test the scalability with accurate calculation of Ψ, which is shown in Fig. 6b. Both results demonstrate our framework scalability in massively parallel NQS training.

5. Conclusion

In this work, we presents QChem-Trainer, a high-performance framework for neural network quantum state (NQS) training, which enables efficient and scalable NQS training on modern HPC platforms. To tackle the scalability

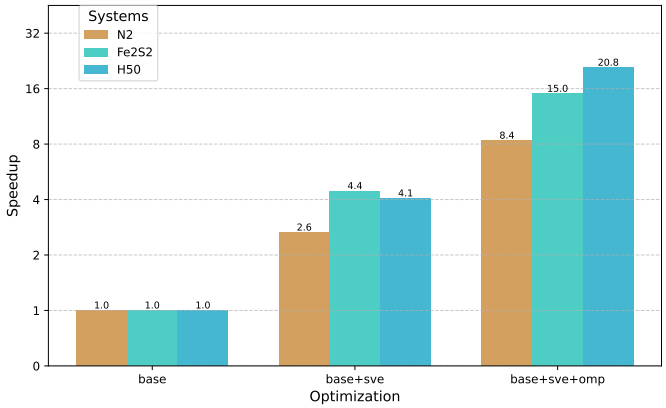
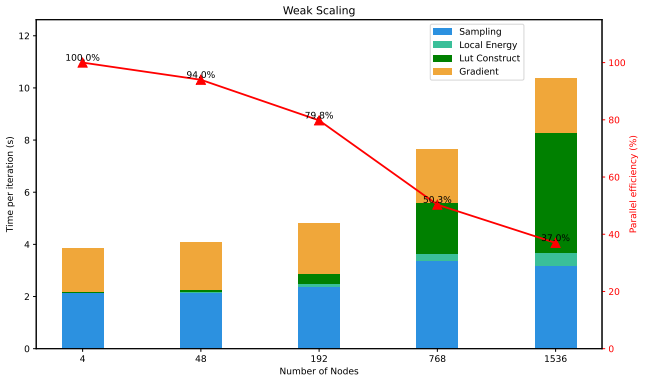
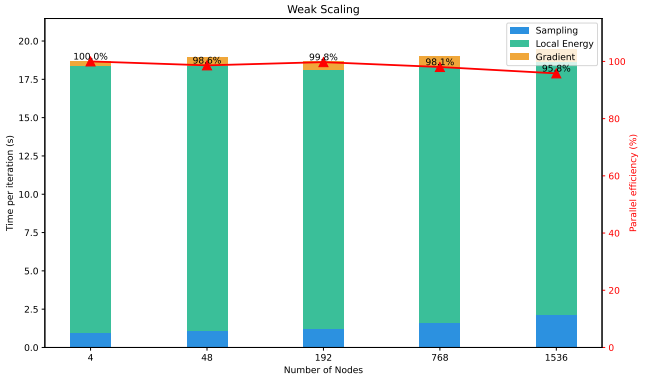


Figure 5: Step by step energy calculation speedup with three chemical systems. Base+sve stands for enabling SIMD optimization while base+sve+omp stands for enabling both of SIMD and thread-level parallel.



(a) Using sample space energy.



(b) Using accurate energy

Figure 6: We test the scaling performance in H_{50} system with two energy calculation methods. Figure (a) demonstrate the scaling result using sample space energy calculation. In contrast, Figure (b) employs another accurate energy calculation method. Both of them showcase the scalability of QChem-Trainer.

and performance bottlenecks in NQS, we propose three key innovations: (i) Scalable and memory-stable sampling parallelism, which overcomes the memory wall and enables NQS training at unprecedented scales; (ii) Multi-level energy calculation parallelism, which fully exploits the parallelism and significantly accelerate the energy evaluation especially for complex molecular systems; and (iii) Cache-centric optimization for transformer-based *ansatz*, which efficiently manages of Key/Value caches to achieve controllable peak memory consumptions. Extensive experiments on real-world chemical systems demonstrate that QChem-Trainer achieves up to 8.41× speedup in overall NQS training. Moreover, strong scaling tests show that QChem-Trainer maintains parallel efficiency of up to 95.8% across 1,526 nodes, highlighting its scalability and efficiency. This work not only paves the way for large-scale *ab initio* quantum chemistry simulations, but also offers practical insights into optimizing such hybrid HPC-AI scientific workloads on next-generation exascale systems.

References

[1] C. D. Sherrill and H. F. Schaefer III, "The configuration interaction method: Advances in highly correlated approaches," in *Advances in quantum chemistry*, vol. 34, Elsevier, 1999, pp. 143–269.

[2] R. J. Bartlett and M. Musiał, "Coupled-cluster theory in quantum chemistry," *Reviews of Modern Physics*, vol. 79, no. 1, pp. 291–352, 2007.

[3] D. Cremer, "Møller–plesset perturbation theory: From small molecule methods to methods for thousands of atoms," *Wiley Interdisciplinary Reviews: Computational Molecular Science*, vol. 1, no. 4, pp. 509–530, 2011.

[4] S. Sorella, "Wave function optimization in the variational monte carlo method," *Physical Review B—Condensed Matter and Materials Physics*, vol. 71, no. 24, p. 241 103, 2005.

[5] G. Carleo and M. Troyer, "Solving the quantum many-body problem with artificial neural networks," *Science*, vol. 355, no. 6325, pp. 602–606, 2017.

[6] J. Hermann, J. Spencer, K. Choo, *et al.*, "Ab initio quantum chemistry with neural-network wavefunctions," *Nature Reviews Chemistry*, vol. 7, no. 10, pp. 692–709, Aug. 2023, ISSN: 2397-3358. DOI: 10.1038/s41570-023-00516-8.

[7] M. Hibat-Allah, M. Ganahl, L. E. Hayward, R. G. Melko, and J. Carrasquilla, "Recurrent neural network wave functions," *Physical Review Research*, vol. 2, no. 2, p. 023 358, Jun. 2020, ISSN: 2643-1564. DOI: 10.1103/PhysRevResearch.2.023358. [Online]. Available: https://link.aps.org/doi/10.1103/PhysRevResearch.2.023358 (visited on 11/28/2022).

[8] D. Wu, R. Rossi, F. Vicentini, and G. Carleo, "From tensor-network quantum states to tensorial recurrent neural networks," *Physical Review Research*, vol. 5, no. 3, p. L032001, 2023.

[9] K. Choo, T. Neupert, and G. Carleo, "Two-dimensional frustrated j 1-j 2 model studied with neural network quantum states," *Physical Review B*, vol. 100, no. 12, p. 125 124, 2019.

[10] J.-Q. Wang, H.-Q. Wu, R.-Q. He, and Z.-Y. Lu, "Variational optimization of the amplitude of neural-network quantum many-body ground states," *Physical Review B*, vol. 109, no. 24, p. 245 120, 2024.

[11] Y. Wu, C. Guo, Y. Fan, P. Zhou, and H. Shang, "Nnqs-transformer: An efficient and scalable neural network quantum states approach for ab initio quantum chemistry," in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, ser. SC '23, Denver, CO, USA: Association for Computing Machinery, 2023, ISBN: 9798400701092.

[12] L. L. Viteritti, R. Rende, and F. Becca, "Transformer variational wave functions for frustrated quantum spin systems," *Physical Review Letters*, vol. 130, no. 23, p. 236 401, 2023.

[13] I. von Glehn, J. S. Spencer, and D. Pfau, "A Self-Attention Ansatz for Ab-initio Quantum Chemistry," *11th International Conference on Learning Representations (ICLR)*, 2023.

[14] *Supercomputer Fugaku - Supercomputer Fugaku, A64FX 48C 2.2GHz, Tofu interconnect D | TOP500*. [Online]. Available: https://www.top500.org/system/179807/ (visited on 02/24/2025).

[15] H. Robbins and S. Monro, "A stochastic approximation method," *The annals of mathematical statistics*, pp. 400–407, 1951.

[16] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," *arXiv preprint arXiv:1412.6980*, 2014.

[17] I. Loshchilov and F. Hutter, "Fixing weight decay regularization in adam," in *International Conference on Learning Representations*, 2019. [Online]. Available: https://openreview.net/forum?id=rk6qdGgCZ.

[18] J. Martens and R. Grosse, "Optimizing neural networks with kronecker-factored approximate curvature," in *International conference on machine learning*, PMLR, 2015, pp. 2408–2417.

[19] T. Zhao, J. Stokes, and S. Veerapaneni, "Scalable neural quantum states architecture for quantum chemistry," *Machine Learning: Science and Technology*, vol. 4, Jun. 2023.

[20] A. Scemama and E. Giner, *An efficient implementation of slater-condon rules*, 2013. arXiv: 1311.6244 [physics.comp-ph]. [Online]. Available: https://arxiv.org/abs/1311.6244.

[21] A. Vaswani, N. Shazeer, N. Parmar, *et al.*, "Attention is all you need," in *Proceedings of the 31st International Conference on Neural Information Processing Systems*, ser. NIPS'17, Long Beach, California, USA: Curran Associates Inc., 2017, pp. 6000–6010, ISBN: 9781510860964.

[22] *HPCG - November 2024 | TOP500*. [Online]. Available: https://top500.org/lists/hpcg/2024/11/ (visited on 02/24/2025).
[23] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, I. Sutskever, *et al.*, "Language models are unsupervised multitask learners," *OpenAI blog*, vol. 1, no. 8, p. 9, 2019.
[24] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," in *International Conference on Learning Representations*, 2019.